42.3 通道集成

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~360k tokens，输出 ~5k tokens（本节）

上一节实现了 MiniClaw 的核心骨架。本节将接入两个消息通道：Telegram Bot 和 WebChat，让用户可以通过不同的渠道与 AI 助手交互。

42.3.1 实现 Telegram Bot 通道适配器

前置准备

1. 在 Telegram 中找到 `@BotFather`，发送 `/newbot` 创建机器人
2. 记录返回的 Bot Token（格式如 `123456789:ABCdefGHIjklMNOpqrsTUVwxyz`）
3. 将 Token 设置到环境变量 `TELEGRAM_BOT_TOKEN`

使用 grammY 框架

grammY 是一个轻量但功能完备的 Telegram Bot 框架（参考 OpenClaw 的 Telegram 通道实现，第 11 章）：

```typescript
// src/channels/telegram.ts

import { Bot } from "grammy";
import { WebSocket } from "ws";

export class TelegramChannel {
  private bot: Bot;
  private gatewayWs: WebSocket;

  constructor(params: {
    botToken: string;
    gatewayUrl: string;
  }) {
    this.bot = new Bot(params.botToken);
    this.gatewayWs = new WebSocket(params.gatewayUrl);
  }

  start() {
    // 等待 Gateway 连接就绪
    this.gatewayWs.on("open", () => {
      console.log("Telegram channel connected to Gateway");
    });

    // 处理 Gateway 返回的事件
    this.gatewayWs.on("message", (raw) => {
      const frame = JSON.parse(raw.toString());
      if (frame.type === "event" && frame.event === "chat") {
        this.handleChatEvent(frame.payload);
      }
    });

    // 监听 Telegram 消息
    this.bot.on("message:text", async (ctx) => {
      const chatId = ctx.chat.id;
      const senderId = ctx.from?.id;
      const text = ctx.message.text;

      // 构造会话键（每个 Telegram 用户一个会话）
      const sessionKey = `telegram:${senderId}`;

      // 发送到 Gateway
      this.sendToGateway("chat.send", {
        sessionKey,
```

```
        message: text,
        channel: "telegram",
        accountId: String(senderId),
        to: String(chatId),
      });
    });

    this.bot.start();
    console.log("Telegram Bot started");
  }

  private handleChatEvent(payload: {
    sessionKey: string;
    state: string;
    message?: { content: string };
  }) {
    if (payload.state !== "final") return;
    if (!payload.sessionKey.startsWith("telegram:")) return;

    // 从 sessionKey 反查 chatId
    //  (实际项目中应维护 sessionKey → chatId 的映射)
    const content = payload.message?.content;
    if (content) {
      // 通过 Gateway 路由回复到正确的 chat
      this.bot.api.sendMessage(/* chatId */, content);
    }
  }

  private sendToGateway(method: string, params: unknown) {
    const frame = {
      type: "req",
      id: crypto.randomUUID(),
      method,
      params,
    };
    this.gatewayWs.send(JSON.stringify(frame));
  }
}
```

通道-会话映射

OpenClaw 使用精心设计的会话键系统（第 6 章）来映射通道用户和会话。MiniClaw 的简化版：

```
// 每个 Telegram 用户 → 一个独立会话
const sessionKey = `telegram:${telegramUserId}`;

// 维护反向映射（用于回复投递）
const channelMeta = new Map<string, {
  channel: "telegram";
  chatId: number;
  userId: number;
}>();
```

42.3.2 实现 Discord Bot 通道适配器

Discord 通道的实现模式与 Telegram 类似（参考第 12 章），但 API 风格不同：

```typescript
// src/channels/discord.ts

import { Client, GatewayIntentBits } from "discord.js";

export class DiscordChannel {
  private client: Client;

  constructor(params: { botToken: string; gatewayUrl: string }) {
    this.client = new Client({
      intents: [
        GatewayIntentBits.Guilds,
        GatewayIntentBits.GuildMessages,
        GatewayIntentBits.DirectMessages,
        GatewayIntentBits.MessageContent,
      ],
    });
  }

  start() {
    this.client.on("messageCreate", async (message) => {
      if (message.author.bot) return;  // 忽略机器人消息

      const sessionKey = `discord:${message.author.id}`;
      // 发送到 Gateway...

      // Discord 支持长消息分段（2000 字符限制）
    });

    this.client.on("ready", () => {
      console.log(`Discord Bot logged in as ${this.client.user?.tag}`);
    });
  }
}
```

Discord 的关键差异：

特性	Telegram	Discord
消息长度限制	4096 字符	2000 字符
富文本	Markdown 子集	Markdown 子集
权限	Bot Token 即可	需要 Intent + Permissions

特性	Telegram	Discord
群聊 @ 触发	默认响应所有 DM	需要 @mention 或配置

42.3.3 实现 WebChat 通道

WebChat 是最灵活的通道——用户通过浏览器直接与 Gateway 的 WebSocket 通信。

前端（HTML + JavaScript）

```html
<!-- ui/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>MiniClaw Chat</title>
  <style>
    body { font-family: system-ui; max-width: 800px; margin: 0 auto; }
    #messages { height: 500px; overflow-y: auto; border: 1px solid #ccc;
                padding: 16px; margin: 16px 0; }
    .user { color: #0066cc; }
    .assistant { color: #333; }
    #input { width: 80%; padding: 8px; }
    button { padding: 8px 16px; }
  </style>
</head>
<body>
  <h1>MiniClaw</h1>
  <div id="messages"></div>
  <input id="input" placeholder="输入消息..." />
  <button onclick="sendMessage()">发送</button>

  <script>
    const ws = new WebSocket("ws://localhost:3000");
    const messages = document.getElementById("messages");
    const input = document.getElementById("input");
    const SESSION_KEY = "webchat:default";
    let reqId = 0;

    ws.onmessage = (event) => {
      const frame = JSON.parse(event.data);
      if (frame.type === "event" && frame.event === "chat") {
        if (frame.payload.state === "final") {
          appendMessage("assistant", frame.payload.message.content);
        }
      }
    };

    function sendMessage() {
      const text = input.value.trim();
      if (!text) return;
      appendMessage("user", text);
```

```
      ws.send(JSON.stringify({
        type: "req",
        id: String(++reqId),
        method: "chat.send",
        params: { sessionKey: SESSION_KEY, message: text },
      }));
      input.value = "";
    }

    function appendMessage(role, content) {
      const div = document.createElement("div");
      div.className = role;
      div.innerHTML = `<strong>${role}:</strong> ${escapeHtml(content)}`;
      messages.appendChild(div);
      messages.scrollTop = messages.scrollHeight;
    }

    function escapeHtml(text) {
      const div = document.createElement("div");
      div.textContent = text;
      return div.innerHTML;
    }

    input.addEventListener("keydown", (e) => {
      if (e.key === "Enter") sendMessage();
    });
  </script>
</body>
</html>
```

静态文件服务

使用 Express 提供 WebChat 前端：

```
// src/channels/webchat.ts

import express from "express";
import path from "node:path";

export function serveWebChat(port: number) {
  const app = express();
  app.use(express.static(path.join(import.meta.dirname, "../../ui")));
  app.listen(port, () => {
    console.log(`WebChat UI available at http://localhost:${port}`);
  });
}
```

流式输出增强

上面的 WebChat 示例只接收最终回复。参考 OpenClaw 的流式设计（第 3 章），可以增加增量更新支持：

```
// Gateway 端：在 Agent 运行时发送 delta 事件
gateway.broadcast("chat", {
  sessionKey,
  state: "delta",
  message: { content: partialText },
});

// 前端：增量追加
if (frame.payload.state === "delta") {
  updateStreamingMessage(frame.payload.message.content);
}
```

通道统一入口

将所有通道的启动逻辑集中管理：

```
// src/index.ts（更新）

// 启动 Gateway
const gateway = new Gateway(3000);
registerRoutes(gateway, sessions, agent);

// 启动通道
if (process.env.TELEGRAM_BOT_TOKEN) {
  const telegram = new TelegramChannel({
    botToken: process.env.TELEGRAM_BOT_TOKEN,
    gatewayUrl: "ws://localhost:3000",
  });
  telegram.start();
}

// 启动 WebChat UI
serveWebChat(8080);

console.log("MiniClaw is running!");
console.log("  Gateway: ws://localhost:3000");
console.log("  WebChat: http://localhost:8080");
```

本节小结

1. **Telegram 通道** 使用 grammY 框架，通过 Bot Token 接收消息，转发到 Gateway，再将 Agent 回复投递回用户。
2. **Discord 通道** 使用 discord.js，需要配置 Intents 和权限，注意 2000 字符消息长度限制。
3. **WebChat 通道** 是最简单的实现——前端直接通过 WebSocket 与 Gateway 通信，无需中间适配层。
4. **通道-会话映射** 以 `{channel}:{userId}` 格式的会话键实现每用户独立会话。
5. **流式输出** 通过 Gateway 的 `delta` 事件实现增量文本更新。